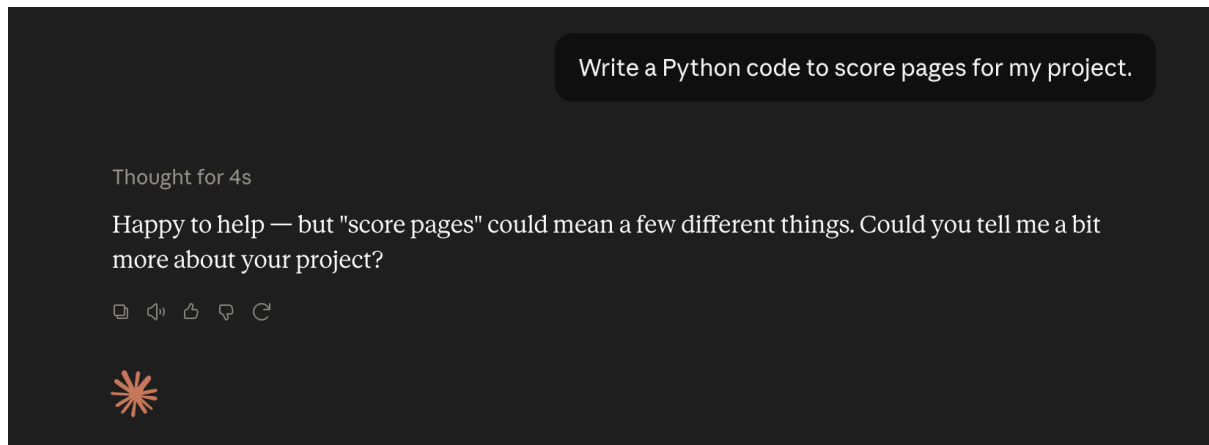


The Prompt Ladder

Baseline

Prompt: Write a Python code to score pages for my project.

Output Excerpt:



What changed in the prompt: N/A.

What actually improved in the output: N/A.

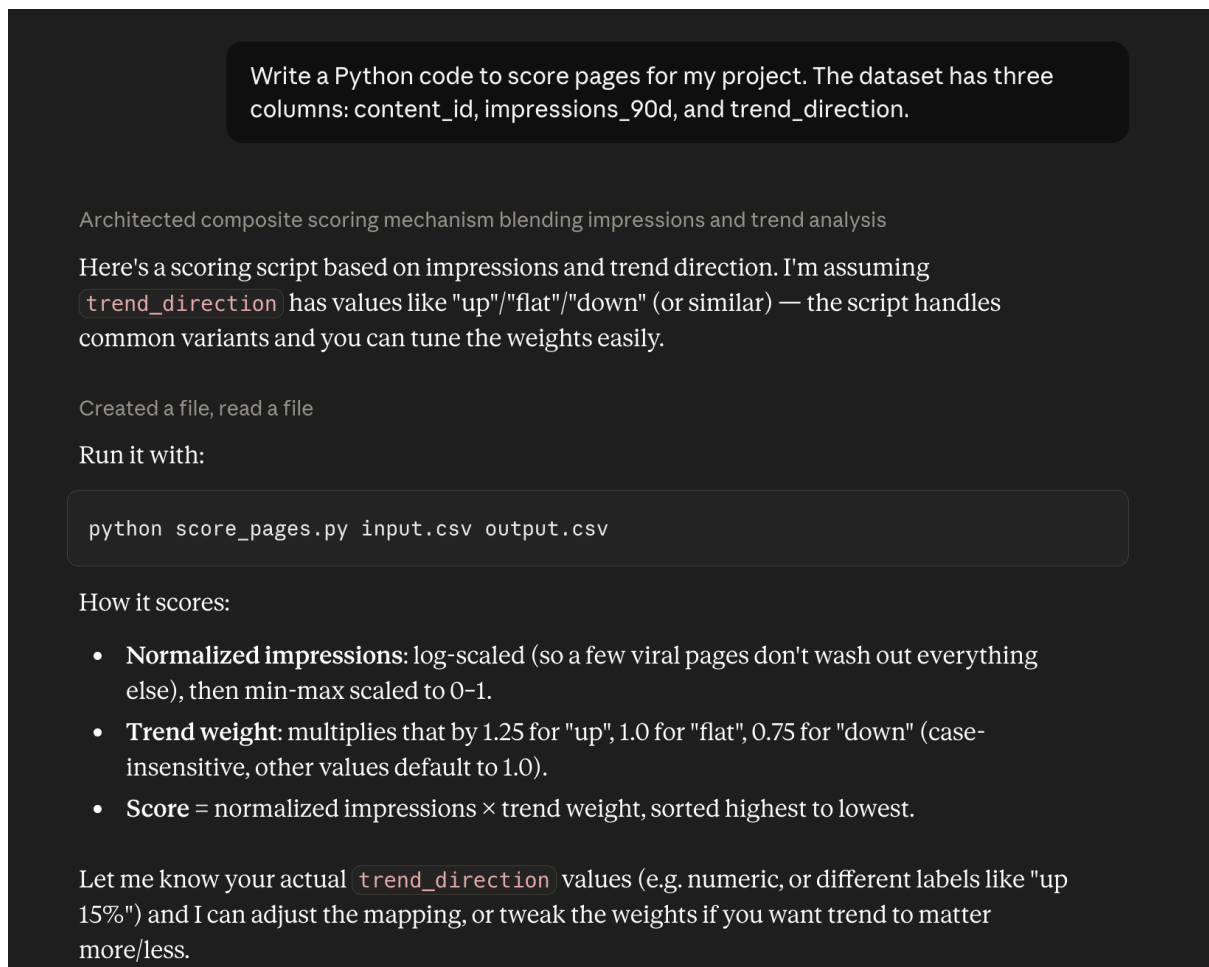
What still failed: The AI generated a random script using fake SEO metrics because it didn't know what my project was. It completely hallucinated the context.

What would you try next: Add real context about my specific dataset.

Version 1

Prompt: Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction.

Output Excerpt:



What changed in the prompt: Added real context (the exact dataset columns).

What actually improved in the output: The AI stopped inventing fake metrics. It grounded its code in the actual columns I provided to structure the logic.

What still failed: The scoring logic itself was completely random. It didn't know what "good" or "bad" meant for my specific usecase, so it just guessed how to weigh the columns.

What would you try next: Add a clearer goal to define how the scoring should actually work.

Version 2

Prompt: Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending.

Output Excerpt:

```
Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending.
```

Architected filtering and sorting logic for decay-ranked pages

Run it with:

```
python rank_decaying_pages.py input.csv output.csv
```

It filters to rows where `trend_direction` is "down" (case-insensitive, whitespace-trimmed), sorts by `impressions_90d` descending, and adds a `rank` column — so page 1 is your highest-traffic page that's currently decaying.

What changed in the prompt: Added a defined goal (filter 'down' trends and sort by impressions).

What actually improved in the output: The script now accurately reflects my actual ML task. It filters the right pages and creates a prioritized action queue instead of generating random scores.

What still failed: The generated code was just one long block of script without any structure. It is not modular or professional.

What would you try next: Add a strict constraint to see if I can force it to be concise.

Version 3

Prompt: Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending. Constraint: Write the entire filtering and sorting logic in exactly one line of code.

Output Excerpt:

Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending. Constraint: Write the entire filtering and sorting logic in exactly one line of code.

Orchestrated filter and sort operations into single pandas expression

Run it with:

```
python rank_decaying_pages_online.py input.csv output.csv
```

The core logic is this single line:

```
python
result = df[df["trend_direction"].str.lower() == "down"].sort_values("impressions_90d", ascending=False)
```

Note: this version assumes `trend_direction` values are already strings (no whitespace-trimming or `.astype(str)` guard like before) — if your data has stray spaces or non-string values, let me know and I can adjust while keeping it one line.

What changed in the prompt: Added a strict constraint (must be one line of code).

What actually improved in the output: This actually made the output much worse. The AI obeyed, but created an unreadable, complex mess.

What still failed: Code readability and maintainability dropped to zero. It is impossible to debug this code in a real production environment.

What would you try next: Remove this bad constraint and replace it with a specified output format to make it professional.

Version 4

Prompt: Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending. Output Format: Structure the code using modular Python functions with type hints and professional docstrings.

Output Excerpt:

Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending. Output Format: Structure the code using modular Python functions with type hints and professional docstrings.

Architected modular solution with typed functions and documentation

Run it with:

```
python rank_decaying_pages_modular.py input.csv output.csv
```

Broken into five functions, each with type hints and a docstring: `load_data` (reads + validates the CSV), `filter_decaying_pages` (keeps only "down" rows), `rank_by_impressions` (sorts + ranks), `rank_decaying_pages` (composes the two), and `save_results` (writes the CSV). `main()` wires them together.

What changed in the prompt: Replaced the constraint with a specified output format (modular functions, docstrings, type hints).

What actually improved in the output: The spaghetti code disappeared. The output looks like it was written by a senior backend engineer, split into logical, maintainable parts.

What still failed: The script defines the functions beautifully but doesn't actually provide a way to test if they work or preview the final dataframe.

What would you try next: Add a verification requirement.

Version 5

Prompt: Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending. Output Format: Structure the code using modular Python functions with type hints and professional docstrings. Verification Requirement: Add a test block at the end that prints the shape of the final dataframe and previews the top 5 rows (Precision@5).

Output Excerpt:

Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending. Output Format: Structure the code using modular Python functions with type hints and professional docstrings. Verification Requirement: Add a test block at the end that prints the shape of the final dataframe and previews the top 5 rows (Precision@5).

Streamlined verification logic by eliminating redundant file operations

Run it with:

```
python rank_decaying_pages_verified.py input.csv output.csv
```

Same modular structure as before, plus a `preview_top_n()` function and a verification block at the end that reloads the saved output and prints its shape plus the top 5 rows.

One honest flag: I labeled it "Precision@5" in your request, but what this prints is really just a **top-5 preview** for a manual sanity check — not a computed precision score. True Precision@5 would need ground-truth labels (e.g., "these are the pages that actually should be prioritized") to compare the ranking against. If you do have such labels, I can add a real Precision@5 calculation instead — just let me know.

What changed in the prompt: Added a verification requirement.

What actually improved in the output: The AI didn't just give me abstract functions; it built a self-testing script. It automatically added a block to measure the dataset size and prove the ranking logic works.

What still failed: Nothing critical. The output is now a complete, testable, and production-ready piece of software.

What would you try next: This is the final version.

Final Version

Prompt: Write a Python code to score pages for my project. The dataset has three columns: content_id, impressions_90d, and trend_direction. The goal is to rank pages that are decaying (trend_direction is 'down'), sorted by their impressions_90d descending. Output Format: Structure the code using modular Python functions with type hints and professional docstrings. Verification Requirement: Add a test block at the end that prints the shape of the final dataframe and previews the top 5 rows (Precision@5).